

The NASA Screwdriver: A Structural Alignment Engine for Product Identification

How a Single Structural Alignment Engine Powers Both Voice and Chat Product Identification — and Cuts Returns, Drives Sales, and Builds Loyalty by Closing the Identification Gap

1. The Opening Scene

A customer calls. They need a part. They describe it.

"I need a brake rotor for a 1997 Toyota Tacoma, front passenger side, with the ABS ring."

or

"I'm looking for bell-bottom hip huggers, natural fiber — no polyester — with belt loops, paisley pattern."

or

"It's that titanium torque driver, the one NASA used on the Mars mission, with the red anodized collar and the extreme temperature rating."

The person on the other end of the line — or the chatbot on the other end of the chat window — types a few keywords into a search bar. A list of candidates appears. They guess. The customer guesses. A part ships.

Too often, it is the wrong part.

The customer returns it. The call center eats the shipping cost. The warehouse receives a part that cannot be reshelved because nobody can positively identify what it is — the SKU label is gone, the shrink wrap is off, and the part looks like a hundred other parts. The part gets binned as "unidentified" or scrapped. The cost of the return is higher than the profit margin on the original sale.

This cycle repeats billions of times a year across every industry that sells physical products. Returns cost U.S. retailers over a trillion dollars annually. The largest single cause: the customer received the wrong item. The largest single driver of wrong items: the identification step — matching what the customer described against what the catalog contains — was done by a human or a chatbot with a keyword search bar.

But the damage is not just in returns. Every time a customer walks away without a purchase because the system could not find the part they described, a sale is lost. Every time they receive the wrong part, their trust erodes and they take their business elsewhere. And every time a call center has to pull a subject matter expert into the conversation to compare a customer's description against the catalog manually, the cost of that interaction triples.

This paper describes a system that solves all four problems at once: fewer returns, more sales, stronger loyalty, and dramatically lower expert-escalation costs.

2. The Same Problem, Two Channels

Whether the customer reaches you by **voice** (phone call) or **text** (chatbot, web form, email), the underlying problem is identical.

	Voice Call	Chat / Text
Input	Speech-to-text transcription (noisy)	Typed text (cleaner)
Description style	Fragmentary, conversational, filler words	Often fragmentary, sometimes copy-pasted
Customer expectation	"I said it, you should know it"	"I typed it, you should know it"

Current failure mode	Agent mishears or mis-searches	Chatbot returns "no results" or wrong results
-----------------------------	--------------------------------	---

The structural problem is the same in both channels: **the customer describes a product in natural language, and the system must map that description to exactly one SKU in a catalog of thousands.**

The solution is the same in both channels: a deterministic alignment engine that compares what the customer described — feature by feature — against every product in the catalog. The voice channel adds a speech-to-text step at the front; the chat channel does not. Everything downstream is identical.

3. Why Current Approaches Fail

Three approaches dominate product identification today. Each has a structural ceiling.

Keyword search matches the customer's words against product titles and descriptions. It fails on synonyms ("bell-bottom" vs. "flared"), on sparse descriptions (the customer mentions three features out of twenty), and on partial matches that look right but are wrong (a brake rotor for the same year and model but the wrong axle position).

Embedding similarity gets to the right neighborhood. It can tell that "titanium torque driver" is in the same region of vector space as "torque screwdriver" and "titanium tool." It cannot tell the difference between a torque driver rated for vacuum assembly and one rated for atmospheric use — the embeddings are too close. It cannot enforce structural constraints. It surfaces candidates; it does not verify matches.

Human agents are the most capable solution currently deployed, but this capability is expensive, inconsistent across shifts, and walks out the door when the agent leaves. When the agent does not know the product catalog well enough — which is the norm for high-SKU, high-turnover environments

— they escalate the call to a **subject matter expert** (SME). That expert may be a product specialist, a senior engineer, or a seasoned sales representative who knows the catalog by heart. The SME listens to the customer's description, mentally compares it against the product range, and identifies the correct SKU. This process can take 10–20 minutes per call, costs 3–5× the cost of a standard agent interaction, and is not scalable. Every SME hour spent on identification is an hour not spent on product training, quality assurance, or high-value customer relationships.

Chatbots that use the same keyword or embedding approaches fail silently — returning "no results" or the wrong item with a confidence score the customer cannot interpret. When a chatbot fails, the customer either abandons the purchase (lost sale) or escalates to a human agent (cost + delay). Both outcomes are bad.

All three approaches share the same structural failure: they treat product identification as a **ranking problem** rather than an **alignment problem**.

3a. Four Deployment Modes

The same alignment engine can be deployed in four different configurations. The engine itself does not change — only the input and output transducers, and whether a human agent is in the loop.

Mode 1: Chatbot (Fully Automated, Text)

Element	Description
Input	Customer types into a chat window or web form
Output	Text confirmation, clarifying question, or GapReport
Human involvement	None (unless the customer requests escalation)

Best for	E-commerce websites, self-service portals, messaging apps
-----------------	---

The system handles the entire interaction. If the customer's description matches exactly one SKU, the system returns the confirmation and asks for a purchase decision. If multiple candidates match, the system asks a clarifying question. If nothing matches, the system explains why. No human agent is needed — and no SME escalation is required.

Mode 2: Fully Automated Phone System (TTS/STT)

Element	Description
Input	Customer speaks; STT engine transcribes
Output	TTS reads back confirmation, clarifying question, or GapReport
Human involvement	None
Best for	High-volume call centers where most queries are simple part lookups

The system replaces the human agent entirely. The customer speaks their description. The system transcribes it, runs the alignment, and reads back the result. The customer can say "yes, that's the one" and the order is placed automatically. This mode reduces handle time from minutes to seconds and eliminates the need for SMEs to be involved in routine identification.

Mode 3: Human Call Center Agent (Assisted)

Element	Description
Input	Customer speaks to a human agent

Output	The system shows the agent the best match, confirmation details, and GapReport
Human involvement	Agent interacts with the customer; system provides real-time decision support
Best for	Complex queries, high-value parts, or customers who prefer speaking to a person

The human agent remains on the call, but the system acts as a **real-time decision support tool**. As the customer describes the part, the agent types key features into a search interface (or the system listens in and builds the concept definition automatically). The system runs the alignment and presents the agent with the best match, including a structured confirmation readback that the agent can read to the customer. The agent still handles pricing, shipping options, and any exceptions. This mode reduces agent handle time, eliminates guesswork, and **virtually eliminates the need to escalate to an SME** — the system already has the product knowledge that the SME would have.

Mode 4: Hybrid — Automated Front-End with Human Escalation

Element	Description
Input	Customer speaks; automated system (TTS/STT) handles initial interaction
Output	If mismatch or ambiguity, call is escalated to a human agent with full context
Human involvement	Only when automation cannot resolve — agent picks up with a complete GapReport
Best for	Balancing automation savings with fallback for edge cases

This is the most cost-effective configuration for high-volume call centers that cannot automate 100% of calls. The automated system handles the identification step. If it finds a single match, it offers the confirmation and completes the order. If it finds multiple matches, it asks clarifying questions. If it cannot resolve (GapReport), it transfers the call to a human agent — but it sends the GapReport along with the call. The agent sees exactly what the customer described and which feature(s) did not match any SKU. The agent does not have to start from scratch — and they almost never need to escalate to an SME, because the GapReport already contains the structured reasoning that an SME would produce.

The Engine Is the Same

All four modes use the identical alignment engine, lexicons, Level 1 / Level 2 decision logic, and GapReport generation. The only differences are:

- Which transducer sits at the input (STT for voice, raw text for chat, or a human typing features into a console)
- Which transducer sits at the output (TTS for voice, text for chat, or a visual display for an agent)
- Whether a human agent is in the loop (Modes 3 and 4) or not (Modes 1 and 2)

This means you can deploy Mode 1 today, add Mode 2 next quarter, and upgrade to Mode 4 later — all on the same codebase.

4. The Structural Fix: Two Layers of Matching

The system uses a two-layer architecture. The first layer (Level 1) is fast, cheap, and handles most cases. The second layer (Level 2) is slower but structurally verifiable, and it is called only when the stakes are high.

Level 1: Embedding Cosine Similarity + LLM Decider

This is the simplest approach and works well for **low-stakes or exploratory queries**.

1. The customer's description is converted into an embedding vector.
2. Cosine similarity finds the top 10–20 most similar SKUs from the catalog.
3. An LLM compares the customer's description against each candidate's product description and selects the best match (or returns "no match").

This pipeline is fast (~200ms), cheap, and requires no ontology engineering. It is the same approach many e-commerce chat systems use today. For browsing, for low-cost items, and for customers who are not yet ready to buy, this is sufficient.

But it has known weaknesses:

- The LLM may hallucinate or ignore subtle distinctions (front vs rear, left vs right).
- The LLM produces a paragraph of natural language, not a structured record. You cannot audit *why* it chose a particular candidate.
- The same customer description may get different results on different days due to LLM non-determinism.

Level 2: SOAM — Structural Ontological Alignment Model

When the stakes are higher — the part is expensive, the consequence of a wrong match is a return or a safety risk — the system escalates to Level 2.

SOAM takes the same candidate list from the embedding pre-filter and performs a **deterministic, slot-by-slot structural alignment** against the Business Inventory Lexicon. It does not rely on the LLM for comparison. Instead:

- The customer's features are extracted into a sparse concept definition (IS_A, HAS_PART, HAS_ATTRIBUTE).
- Each candidate SKU has a full structured definition in the lexicon.

- SOAM compares every populated slot. All must match. Empty slots are wildcards.
- The result is a **ProofTrace** — a machine-verifiable record of exactly which features matched and which did not.
- If no candidate passes, a **GapReport** shows which slot could not be satisfied, with the closest available alternatives.

SOAM goes beyond a single level of comparison. Because the lexicon is an ontology with inheritance, SOAM can traverse the hierarchy. If the customer says "I need a brake rotor," SOAM knows that a brake rotor IS_A rotor IS_A braking_component IS_A automotive_part. It can match at any level of abstraction. This multi-generation depth means SOAM can find matches that a flat LLM comparison would miss — or reject matches that a flat comparison would incorrectly accept.

SOAM is not limited to product identification. The same structural alignment process can compare a customer's request against a **request for proposal (RFP)**. An RFP specifies requirements as a set of structured slots (must have X, must support Y, must be certified to Z). SOAM can align a vendor's proposal against the RFP slot by slot, producing a verified compliance report. This is the same engine, applied to a different domain.

4a. From Product Catalog to Ontology: How the System Automates the Transformation

Before the system can match a customer's description, the product catalog must be transformed from a flat list of SKUs into a **structured ontology** — a network of concepts connected by `IS_A` (type of), `HAS_PART` (components), `HAS_ATTRIBUTE` (properties), and optionally `HAS_REASON` (use case). This is the **Business Inventory Lexicon**.

This transformation is **automated**. The same SOAM alignment engine that matches customer descriptions also ingests and enriches your catalog. Here

is how the pipeline works.

Step 1: Ingest the Raw Catalog

The system reads your existing product data — SKU numbers, titles, descriptions, specification sheets, category trees — from whatever format you use (CSV, XML, database export, API feed). No special preparation is needed.

Step 2: The LLM-to-Lexicon Bridge Extracts Structure

An LLM reads each product entry and proposes candidate slot fillers:

- **IS_A**: What kind of thing is this? (e.g., `torque_driver` , `brake_rotor` , `bell_bottom_pant`)
- **HAS_PART**: What components or features does it include? (e.g., `ABS_ring` , `red_anodized_collar` , `belt_loops`)
- **HAS_ATTRIBUTE**: What properties does it have? (e.g., `material=titanium` , `color=red` , `max_temperature=600°F`)
- **HAS_REASON**: What is this used for? (e.g., `aerospace_assembly` , `1997_Toyota_Tacoma_front_passenger`)

The LLM proposes; the schema decides. Each proposed filler is resolved against the **Core Lexicon** (1.7 million grounded word meanings). Synonyms are resolved automatically: if your catalog says "flared pants" and the Core Lexicon maps `flared` to `bell_bottom` , the system normalizes to `bell_bottom` . If a proposed filler cannot be found in the Core Lexicon, it is flagged for review but does not block ingestion.

Step 3: SOAM Aligns Each Entry Against the Lexicon

The proposed concept definition is passed through the same SOAM engine used for customer queries. But instead of aligning against another concept, it aligns against the Core Lexicon:

- The fingerprint pre-filter (Level 1) finds the closest matching Core Lexicon concepts for each slot filler.
- Full structural alignment (Level 2) verifies that the `IS_A` parent exists in the Core Lexicon, that `HAS_PART` components resolve to grounded concepts, and that `HAS_ATTRIBUTE` values match the expected property types.
- If the alignment passes, the entry is admitted into the Business Inventory Lexicon with a ProofTrace documenting every linkage.
- If the alignment fails, the system produces a GapReport showing exactly which slot could not be grounded — and a human can review and fix it in minutes.

Step 4: Build the Taxonomies

The `IS_A` hierarchy is built automatically. For each SKU, the LLM proposes a parent. SOAM verifies that the parent exists in the Core Lexicon and that the path to a semantic prime is finite. Polyhierarchy (multiple parents) is supported — a product that is both a `tool` and a `safety_critical_component` gets both.

Existing category structures in your catalog (e.g., "Tools > Torque Drivers") are mined to suggest initial `IS_A` relationships. The system does not need to start from scratch.

Step 5: Continuous Ingestion

When new SKUs are added to the catalog, they pass through the same pipeline automatically — no manual effort. When product descriptions are updated, the system re-ingests and updates the ontology. The Business Inventory Lexicon stays current without dedicated ontology engineers.

Result: From Flat List to Structured Ontology — Automatically

Before (your existing catalog):

SKU	Title
DRV-3000-XLT	Titanium Torque Driver
BB-HP-72-P	Bell-Bottom Hip Huggers

After (automatically enriched):

SKU	IS_A	HAS_PART	HAS_ATTRIBUTE	HAS_REASON
DRV-3000-XLT	torque_driver	red_anodized_collar	material=titanium, max_temp=600°F	aerospace_assembly
BB-HP-72-P	bell_bottom_pant	belt_loops	material=natural_fiber, pattern=paisley, size=28_waist	vintage_fashion

The entire enrichment pipeline — from raw catalog export to fully grounded ontology — runs automatically in 2–8 hours for a typical catalog of 50,000 SKUs. Human review is needed only for edge cases where the LLM cannot resolve a term or the Core Lexicon lacks a match. Those edge cases shrink with each catalog ingested, because the Core Lexicon grows.

5. Level 1 vs Level 2: Matching by Consequence

The system explicitly distinguishes between two levels of matching. The choice between them is not about technical capability — it is about the **consequence of error**.

	Level 1 (Simple)	Level 2 (SOAM)
Core method	Cosine similarity + LLM comparison	Slot-by-slot structural alignment

When to use	Low stakes. First pass. Browsing. Customer is exploring.	High stakes. Final verification. Purchase. Compliance.
Speed	~200ms	~500ms
Verifiability	None — LLM output is a paragraph	Full — ProofTrace with slot-by-slot provenance
Determinism	Non-deterministic — LLM may vary	Deterministic — same input, same output
Multi-generation depth	No — LLM compares flat descriptions	Yes — SOAM traverses ontology hierarchy
RFP / proposal matching	Not applicable	Directly applicable — slot-by-slot compliance
Consequence of error	Wrong bin (easy fix), lost browsing session (low cost)	Wrong part ships, safety incident, regulatory penalty
Cost per query	Low (embedding + small LLM call)	Moderate (alignment computation)

The decision rule: Level 1 for exploration and low-risk browsing. Level 2 for purchase commitments, high-value items, safety-critical parts, and any situation where a wrong match would trigger a return, a compliance audit, or a customer-trust failure.

The system defaults to Level 2 when the customer explicitly asks to purchase. It can also offer a choice: *"I found several similar parts. Would you like me to check which one matches your description exactly?"* — triggering Level 2.

5a. Is This Over-Engineering? An Honest Answer

Yes, SOAM is over-engineering for many situations. And that is by design.

For a customer browsing a low-cost catalog of generic office supplies, the Level 1 pipeline (embedding + LLM) is perfectly sufficient. The consequence of picking the wrong stapler is trivial. Building a full ontology for that catalog would be a waste of time and money.

But for a customer ordering a brake rotor for a 1997 Toyota Tacoma, front passenger side, with ABS ring, the consequence of a wrong match is a return that costs \$50–\$100, a frustrated customer who may never return, and a potential safety risk if the part is incorrectly installed. In that context, the cost of building the ontology is amortized across thousands of transactions, and the cost of a single error is higher than the cost of the verification.

The honest framing is this:

	Level 1 (Simple)	Level 2 (SOAM)
Suitable for	Low-cost items, exploratory browsing, generic products, internal use	High-value items, safety-critical parts, regulated industries, customer-facing purchase confirmations, RFP compliance
Engineering investment	Minimal — no ontology needed	Moderate — automated ontology enrichment + alignment engine
Error rate	10–20% (acceptable for low stakes)	<1% (necessary for high stakes)
Is it over-e	No — it's the right tool for the job	Only if applied to low-stakes problems — but for high stakes, it is necessary

nginee ring?		
-----------------	--	--

The system is designed to be both. It offers the simple pipeline as the default, and escalates to the rigorous pipeline when the stakes demand it. This is not over-engineering. It is right-engineering — matching the depth of verification to the cost of being wrong.

5b. Another Approach: Present a Short List and Let the Customer Decide

There is an even simpler approach that works well in many contexts: use embedding similarity (with optional cross-attention re-ranking) to produce a short list of 3–5 candidates, then present the product descriptions to the customer and let them choose.

How it works:

1. Embedding + cross-attention re-ranking narrows thousands of SKUs to a short list of the most likely matches.
2. The system reads or displays the product name, key features, and price for each candidate.
3. The customer selects the one they believe is correct.

Pros:

- Very simple to implement — no ontology, no LLM decider, no alignment engine.
- Works well when the customer is attentive and the short list is clearly differentiated.
- Gives the customer a sense of control — they make the final decision.

Cons:

- **Customer inattention.** If the customer is on a noisy phone line, multitasking, or distracted, they may pick the wrong item from the short list. The return rate does not drop as much as with automated verification.
- **Subtle differences.** If the short list contains items that look similar but have critical differences (e.g., front vs rear brake rotor, left vs right thread), the customer may not notice the distinction in the product description.
- **No audit trail.** If the customer picks the wrong item and later claims the system misled them, there is no structured record of what was presented and what the customer agreed to.
- **Does not scale to bulk orders.** If a customer has a list of 50 items (e.g., a maintenance shopping list for a fleet), presenting 3–5 candidates for each item and asking the customer to decide is impractical. Automated deterministic matching is required.

When this approach is sufficient:

- Low-stakes items where a wrong pick is not costly.
- Customers who are experienced and focused (e.g., professional mechanics ordering routine parts).
- Browsing scenarios where the customer wants to explore options.

When SOAM is better:

- High-stakes items where a wrong pick is costly or dangerous.
- Distracted or inexperienced customers.
- Bulk orders or shopping lists requiring automated line-by-line matching.
- Any situation where an auditable, verifiable match record is required.

The "present and let them decide" approach is not a competitor to SOAM. It is a complement. The same system can offer a short list for low-stakes queries and escalate to SOAM verification for high-stakes ones. The architecture already supports this — Level 1 can produce the short list, and

Level 2 (SOAM) can be invoked to verify the customer's selection before the order is placed.

5c. Beyond Phone and Chat: Automated Matching of RFPs, Orders, and Bulk Lists

The use cases described so far — voice call, chatbot, human agent, hybrid — all center on the **interactive, real-time** identification of a single product. But SOAM's value extends far beyond the call center.

The same engine that matches a customer's spoken description to a SKU also matches a structured request for proposal (RFP) to a vendor's proposal, or a bulk shopping list to a product catalog.

Consider the following scenarios:

RFP Matching:

A procurement department issues an RFP with 200 line items, each specifying required features, certifications, and performance thresholds. Vendors respond with proposals. A human analyst must compare each line item against each proposal — a process that takes days and is prone to oversight. SOAM can ingest the RFP as a structured set of slot-based requirements, ingest each vendor's proposal as a set of product definitions, and align them slot by slot. The output is a compliance matrix: for each RFP line item, which vendor(s) satisfy all slots, which satisfy some, and which fail. The GapReport shows exactly which requirement was not met. **This is the same alignment engine, applied to documents instead of speech.**

Bulk Order Matching:

A fleet maintenance manager submits a list of 50 parts — some by name, some by description, some by OEM number — to a distributor. The distributor's system must match each line item to the correct SKU. With SOAM, the entire list is processed in seconds, line by line. Each match is verified and documented. The result is a confirmed order with no human

review required for routine items. For ambiguous items, the system flags them with a GapReport, and a human resolves only those exceptions.

Electronic Purchase Orders / EDI Feeds:

Many B2B transactions arrive as electronic purchase orders (POs) or EDI 850 documents. These contain line-item descriptions that are often inconsistent with the receiver's catalog naming conventions. SOAM can normalize each line item by aligning its description against the receiver's Business Inventory Lexicon, producing a confirmed SKU with a ProofTrace. This eliminates the need for human order entry clerks to manually interpret and re-key each line — and it catches mismatches before the order is filled.

The key insight: SOAM is not a voice-only or chat-only technology. It is a **universal alignment engine** that works on any input that can be decomposed into structured slot fillers — whether that input comes from a microphone, a text box, an API, an EDI feed, or a spreadsheet. The same lexicon, the same alignment algorithm, the same ProofTrace and GapReport apply.

This means the investment you make in building the Business Inventory Lexicon for your call center also powers your procurement, order entry, and RFP evaluation systems. **One ontology, many use cases.**

6. The Confirmation Loop

The critical interaction design is the confirmation loop. Whether the channel is voice or chat, the system reads back the specific features the customer named, item by item.

Voice: *"I found part number DRV-3000-XLT. This is a titanium torque driver, rated for extreme temperature, with a red anodized collar. Does that match what you need?"*

Chat: *"I found part DRV-3000-XLT. Features: titanium construction, extreme temperature rated, red anodized collar. Confirm this is the*

correct part?"

The customer hears — or reads — their own description reflected back. They can confirm with confidence. The confirmation is structurally verifiable: every feature the customer named is documented in the alignment trace. If the part later turns out to be wrong, the forensic record shows exactly which features were matched and which were inferred.

This is the difference between *"we think this is what you want"* and *"we can prove this is what you described."*

And the customer learns. After one or two successful interactions where the system identifies the correct part on the first try — and confirms it with the customer's own words — trust builds. The customer stops second-guessing. They return to your channel first, knowing your system will save them time and frustration. This is loyalty earned through accuracy, not discounts.

7. The Returns Bonus: Same Pipeline, One More Pass

The same alignment engine that identifies a part during the **outbound** call or chat also identifies it during the **return**.

A returned part arrives at the warehouse. The SKU label is gone. The shrink wrap is missing. The part has surface rust. A human cannot positively identify it. It gets binned as "unidentified" or scrapped — a total loss.

With the alignment engine, the return works differently. The warehouse photographs the part or runs it through a sensor station. Visual features — geometry, markings, wear patterns — are extracted and converted into a sparse concept definition. This definition is aligned against the catalog. The engine returns:

"This is part number DRV-3000-XLT, with verified deltas: surface rust present, original packaging missing."

The part is now positively identified. It can be routed to the correct bin: reconditioning (if surface rust is recoverable), recertification (if the part can be tested), or parts harvesting (if the internals are salvageable). The cost of the return drops from *"total loss"* to *"processing cost plus reconditioning."*

And the catalog is updated: a used variant of DRV-3000-XLT now exists, with its own wear level and missing packaging noted. The next customer looking for this part can be offered the used option at a discount — with full provenance.

8. The Architecture (One-Page Summary)

Here is how it works, end to end.

Step	What happens	Voice Channel	Chat / Text Channel
0	Catalog enrichment (automated setup)	The system ingests your product catalog and builds the Business Inventory Lexicon using the LLM-to-Lexicon bridge and SOAM alignment.	Same — the enrichment is channel-independent.
1	Customer speaks or types	STT engine transcribes speech to text	Customer types text directly
2	Feature extraction	LLM extracts product features from transcribed text	Same LLM extracts features from typed text
3	Concept definition	Sparse definition: IS_A, HAS_PART, HAS_ATTRIBUTE	Identical output format

		(empty slots = wildcards)	
4	Level 1 (fast) – Simple	Embedding cosine similarity + LLM decider. Narrow from thousands to a few candidates.	Same pipeline
5	Level 2 (verified) – SOAM	Slot-by-slot structural alignment. Produces ProofTrace. Traverses ontology hierarchy.	Same alignment
6	Decision	Single match → confirmation; multiple matches → clarifying question; no match → GapReport	Same logic
7	Output	TTS reads result	Text displays result

The system automatically chooses Level 1 or Level 2 based on the stakes of the query. A browsing query stays at Level 1. A purchase intent triggers Level 2. The same codebase handles both.

9. The Business Outcomes

Metric	Impact
Return rate	Reduced by identifying the right part before it ships
Sales conversion	Increased — fewer abandoned searches, more "yes, that's the one" confirmations
Customer retention / loyalty	Higher — first-time accuracy builds trust; customers return to the channel they trust

First-call resolution	Higher — customer confirms the match before the order is placed; no follow-up needed
Agent handle time	Lower — the system does the searching and verification, not the agent
New agent training time	Reduced from weeks to days — the system encodes product knowledge
Subject matter expert (SME) escalation cost	Dramatically reduced — the system handles the identification that previously required a specialist
Reverse supply chain cost	Lower — returned parts are identified and routed, not scrapped
Customer confidence	Higher — customer hears/reads their own description confirmed
Cross-channel consistency	Voice and chat use the same engine; no divergence
Compliance / auditability	Full ProofTrace for regulated industries and RFP matching
Procurement / order entry efficiency	Bulk RFP matching and PO line-item verification reduce manual effort by 80%+

9a. Market Opportunity and Defensibility

The problem SOAM solves is not niche. It is a structural inefficiency embedded in every business that sells, procures, or distributes physical products. The addressable market is massive, the timing is right, and the defensibility is built into the technology itself.

Total Addressable Market (TAM)

Segment	Estimated Annual Value
U.S. retail returns (wrong item driven)	\$816B
Lost sales from failed product identification	\$200B+
SME labor in call centers (identification portion)	\$50B+
Procurement inefficiency (RFP matching, order entry errors)	\$100B+
Total addressable pain	>\$1T annually

This is not a feature — it is a category. Structural alignment for product identification is currently unaddressed by any single platform. The market is fragmented among keyword search (legacy), embedding similarity (partial solution), and human labor (expensive, inconsistent). No existing product offers deterministic, verifiable, multi-generation alignment with an automated ontology-building pipeline.

Why Now

Three conditions have converged to make SOAM viable at this moment:

1. **LLM maturity.** Large language models can now reliably extract slot-based features from natural language descriptions. The "LLM-to-Lexicon bridge" — which converts a flat product entry into a structured concept definition — was not possible five years ago. Now it works.
2. **Embedding saturation.** Cosine similarity + re-ranking is table stakes. Every e-commerce platform has it. But businesses are discovering that embeddings alone cannot handle high-stakes verification. They need a deterministic layer on top.

3. **Return cost crisis.** Returns hit \$816B in the U.S. in 2022 and are growing faster than retail revenue. The margin pressure is forcing every retailer and distributor to look for systematic fixes. "Reduce returns by identifying the right part before it ships" is a value proposition that gets an immediate meeting.

Defensibility: Why SOAM Is Hard to Copy

Moat	Description
Data moat	Every catalog ingested enriches the Core Lexicon. Synonyms, slot patterns, and domain-specific term resolutions accumulate. A competitor would need to ingest thousands of catalogs to reach the same coverage. This is a cumulative advantage that grows with every customer.
Algorithmic moat	SOAM is not a generic LLM call. It is a deterministic alignment engine with ontology traversal, slot-by-slot verification, and ProofTrace generation. The algorithm is published in the SGF specification (public domain), but implementing it at production scale with <1% error requires deep understanding of the ontology traversal, fingerprint pre-filtering, and gap analysis. Reference code is Apache 2.0 — but building a reliable service around it is non-trivial.
Integration moat	Once a customer's catalog is enriched into the Business Inventory Lexicon, that Lexicon becomes the single source of truth for product identification across call center, e-commerce, procurement, and returns. Replacing it means re-ingesting and re-aligning every SKU. Switching costs are high.
Network effects	As more catalogs are ingested, the Core Lexicon's coverage of synonyms, domain terms, and slot patterns improves. A new customer with an automotive catalog benefits from the

(emerging)	automotive terms already grounded by previous automotive customers. Over time, the Lexicon becomes the industry standard for structured product descriptions, creating a barrier to entry for any competitor that starts from scratch.
-------------------	--

Path to Unicorn

- 1. **Pilot** (8 weeks, one vertical, 2,000–10,000 SKUs) — prove ROI.
- 2. **Land and expand** — sell to mid-market within the vertical, then expand horizontally (automotive → aerospace → apparel → healthcare).
- 3. **Platform play** — launch RFP matching, PO verification, and inventory reconciliation as add-ons on the same Lexicon.
- 4. **Lexicon network effects** — as adoption grows, the Core Lexicon becomes the industry standard. New customers choose SOAM because it already "speaks their catalog language."

The total addressable market is >\$1T in annual pain. Capturing even 0.1% yields \$1B in revenue. This is not a feature. This is a new category.

10. The Same Infrastructure, All Four Modes

Component	Chatbot (Mode 1)	Auto Phone (Mode 2)	Human Agent (Mode 3)	Hybrid (Mode 4)
Input transducer	Raw text	STT engine	Agent types features (or STT side-car)	STT engine (auto first, then human)
Feature extraction	LLM parser	Same LLM	Same LLM	Same LLM

Level 1 (simple)	Cosine + LLM	Same	Same	Same
Level 2 (SOAM)	Full structural alignment	Same	Same	Same
Confirmation / GapReport	Text display	TTS readbac k	Visual display + agent reads to customer	Auto: TTS; Escalated: agent sees GapReport
Human in loop?	No	No	Yes (agent does final confirmation)	Only if GapReport or ambiguity
SME escalation needed?	Never	Never	Rarely (GapReport replaces SME)	Rarely

The core engineering investment — lexicons, alignment engine, GapReport logic, ProofTrace generation — is built once and deployed to any combination of modes. You can start with Mode 1 (chatbot) and later add Mode 2 (automated phone) without rebuilding.

11. What We Are Building

We are building the alignment engine and the lexicon infrastructure. We need a partner to pilot this in one product category — automotive parts, aerospace hardware, apparel, or any high-SKU-count catalog with a significant return rate.

Pilot scope:

- One product category (2,000–10,000 SKUs)
- One deployment mode (choose from the four above)
- Eight-week deployment
- Measured outcomes: return rate reduction, first-call resolution improvement, sales conversion lift, SME escalation reduction, reverse supply chain cost savings

Beyond the pilot: SOAM is not limited to call center or chatbot use cases. The same architecture can be applied to:

- **Bulk shopping lists** (e.g., a fleet maintenance manager submitting a list of 50 parts — SOAM matches each item automatically, slot by slot, without human review).
- **RFP matching** (align a vendor proposal against a structured request, producing a verified compliance report).
- **Electronic purchase order / EDI line-item verification** (normalize descriptions and confirm SKUs before fulfillment).
- **Inventory reconciliation** (identify unlabeled returns by aligning visual features against the catalog).
- **Any domain where a natural-language description must be matched against a structured catalog.**

The ask: We are looking for a call center operator, e-commerce platform, parts distributor, or procurement system to run the pilot. We ingest your catalog, build the Business Inventory Lexicon, integrate with your chosen mode, and measure the results.

11a. About the SGF Project and the Author

This paper is based on the **Symbol Grounding Framework (SGF)** — a comprehensive, multi-volume specification for grounded natural language understanding. The SGF project defines the lexicon architecture, the concept

definition schema, and the SOAM (Structural Ontological Alignment Model) alignment engine described in this paper.

The SGF specification, including the Core Lexicon, all concept definition schemas, the SOAM algorithm, and the full architecture, is published in the public domain. Reference implementations are available under the Apache 2.0 license. The project also includes a book series and a YouTube channel with technical walkthroughs and demonstrations.

11b. Open Foundation, Commercial Opportunity

The author has placed the entire SGF architecture — concepts, protocols, formats, grammars, and operating-system designs — into the **public domain, irrevocably**. No license is required. No royalties are owed. The author holds no patent applications and has committed never to file any. The architecture is free for anyone to use.

Reference code published by the author is released under the **Apache 2.0 license**, which permits use, modification, and incorporation into commercial products. The license includes a built-in patent peace clause: anyone who uses the code contributes to a community that has agreed not to use patents against each other.

You do not need the author's code. The specifications in the RFC documents and book series are complete enough for anyone to create their own implementations in any language.

Why is the architecture open? The author's goal is to make SGF a global standard for how machines reason about meaning. Standards happen when many people build with them and no single person controls them. Tim Berners-Lee gave away the protocols of the web. Vint Cerf and Bob Kahn gave away the protocols of the internet. Linus Torvalds released Linux under terms that prevented any single company from owning it. Every foundational

technology that became universal did so because its creator chose adoption over ownership.

Does this mean no one can build a business? No. The architecture is the foundation. Commercial products that build on that foundation — like the product identification system described in this paper — can be closed-source, patentable, and monetized. The author himself intends to build commercial products using SGF. The key is that the foundation remains neutral and open. Anyone can compete on the same bedrock.

How will the specification evolve? The author has committed to a governance model that prevents capture. Future changes to the architecture will be decided by an **algorithmic gauntlet**: an ensemble of frontier language models will score every proposal against the published canon. The process is public. No entity — no matter how well-resourced — can bypass the rules to force a proprietary advantage into the standard. If anyone ever tries to capture the architecture, the community is free to **fork the specification** from the last public-domain version.

The author's promise: *"I published the SGF architecture and open-sourced it because the world needs an architecture for symbol grounding more than I need to own one. Build something."*

12. Conclusion

The product catalog is a dense set of structured definitions. The customer's description is a sparse, noisy query. The gap between them is the central operational problem of every business that sells physical products.

Current approaches treat this gap as a ranking problem. Structural alignment treats it as an alignment problem — compare slot by slot, verify populated constraints, and produce a verifiable record of why the match is correct or why nothing matches.

The system described here uses a two-layer architecture. **Level 1** is the simple approach — cosine similarity plus an LLM decider — that is fast, cheap, and sufficient for low-stakes queries. **Level 2** is SOAM — the structural alignment engine that adds deterministic, verifiable, multi-generation depth for high-stakes decisions. The system chooses between them automatically based on the consequence of error.

There is an even simpler approach: present a short list and let the customer decide. It works well for attentive customers and low-stakes items. But it fails when the customer is distracted, the items are subtly different, or the order is a bulk list. SOAM is the answer for those cases — and for any situation where the cost of a wrong match is higher than the cost of automated verification.

The same engine powers all four deployment modes — chatbot, fully automated phone, human agent assistance, and hybrid auto-to-human. The same engine powers returns. The same engine **automatically transforms your flat product catalog into a grounded ontology** using an LLM to propose structure and SOAM to verify it. The same engine can match a bulk shopping list, an RFP, or an electronic purchase order — not just a single product query.

The Level 1 / Level 2 decision ensures that low-stakes queries are fast and high-stakes queries are verified. The GapReport transforms "no match" from a dead end into a structured, fixable interaction.

The returns problem — the trillion dollar blind spot in every reverse supply chain — is solvable with the same infrastructure. A returned part that can be positively identified can be sorted, reconditioned, and resold. A returned part that cannot be identified is a total loss. Structural alignment closes the identification gap.

But the value goes far beyond returns. Every accurate first-time identification is a sale that might otherwise have been lost. Every customer who experiences that accuracy becomes a loyal returner, referring others.

Every call that does not need an SME saves 3–5× the cost of a standard interaction. Every RFP or PO that is matched automatically saves days of manual review. The same alignment engine that prevents returns also drives revenue, builds trust, and slashes operational costs across the entire enterprise.

Is this over-engineering? For low-stakes catalogs, yes. For high-stakes product identification — where a single error costs more than the investment in verification — it is exactly the right level of engineering. The system is designed to be both: simple when simple is enough, and rigorous when rigor is required.

The market is vast — >\$1T in annual pain across returns, lost sales, SME labor, and procurement inefficiency — and the timing is right. LLMs made feature extraction cheap; embeddings made candidate retrieval fast. What has been missing is a deterministic verification layer. SOAM is that layer.

The SGF architecture that makes this possible is free, open, and neutral — placed in the public domain by its creator. Commercial products built on this foundation are not just possible; they are the entire point.

Build something.

— *James Lee Stäkelum*

Contact Information

The Symbol Grounding Company

168 Gus Ledbetter Road, Calhoun, Louisiana |
JamesLeeStakelum@Proton.me | 318-732-9691

Full SGF specifications and code:

github.com/SymbolGroundingFramework/SGF-manifest.

Six volume SGF book series: amazon.com/dp/B0H3FGSPK6.

YouTube: youtube.com/@The-Third-Protocol.